

How your lakehouse design decisions affect Direct Lake performance



Nikola Ilic

Data Mozart | Microsoft Data Platform MVP



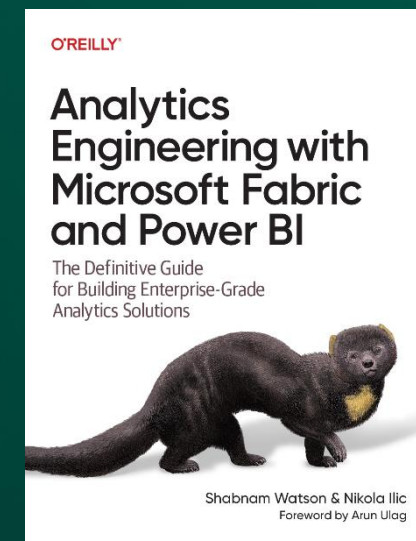
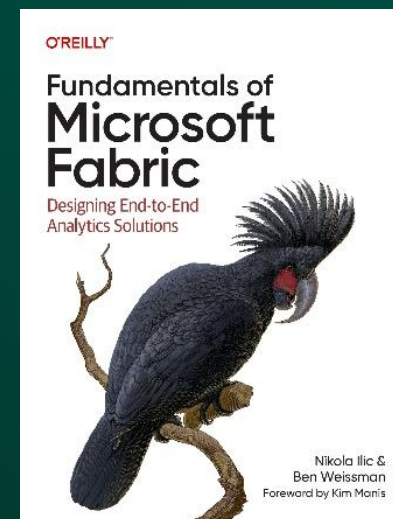
Nikola Ilic

Principal Architect



data-mozart.com

- *I'm making music from the data!*
- Power BI and Fabric addict, blogger, speaker...
- Father of 2, **Barca** & Leo Messi fan...



Direct Lake - The promise and the fine print

The promise

One copy. Reports read the Delta tables directly, no import copies to feed.

Import speed. Warm queries run on the same VertiPaq engine as import mode.

No refresh pipeline. A model refresh is framing, not a data copy.

The reality in the field

Slow first queries. Someone pays a cold start every morning.

Queries that fail. Exceeding a guardrail breaks the visual (or forces a DirectQuery fallback).

Performance regressions. Reports slow down after an ETL redesign.

The root cause usually is not the semantic model. It is how the lakehouse tables are written.

Two flavors, two failure modes



Direct Lake on SQL

Single source

Can fall back to DirectQuery: guardrails, T-SQL views, unsupported features.

Failure mode: A sudden latency cliff when a query switches engines unannounced.

Detect it: Query plans in Performance Analyzer or DAX Studio.



Direct Lake on OneLake

Multi source

Never falls back to DirectQuery.

Failure mode: Degradation through file layout, transcoding cost, and memory.

Detect it: Segment residency and Delta history, live in this session.

Today is about the second failure mode. It applies to both flavors.

PART 1

How does Direct Lake actually work?

Framing – Direct Lake “refresh”

- A refresh re-points the model to a Delta table version
- Metadata only: fast, cheap, decides which in-memory segments are still valid



Framing in a Direct Lake semantic model

Framing initiated via automatic update or manual, scheduled, or programmatic model refresh

2



Semantic model (Direct Lake)

Each table binds to the Parquet files, referenced by a particular Delta log commit

1



OneLake

stores metadata and Parquet files

3

Last framing operation



Delta tables

comprise Parquet files

4



Parquet files added before *last* framing



Parquet files added after *last* framing

New framing operation



Parquet files added after *new* framing

5

6

Framing sets the baseline for all future transcoding events

Subsequent data modifications aren't visible until a new framing operation occurs

At the end of framing, resident columns may be evicted from memory

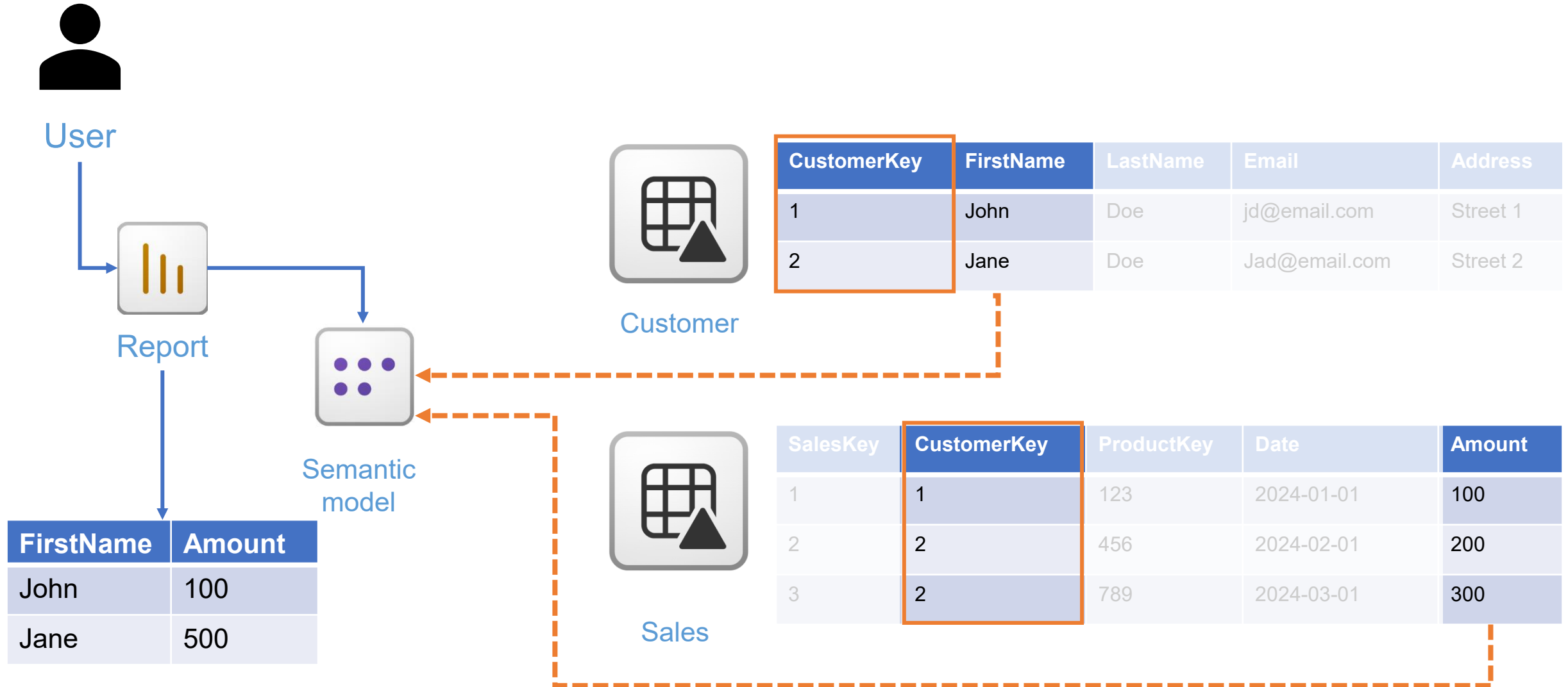
7

[Diagram sourced from MS Learn](#)

Transcoding – Loading columns into memory

- On first touch, parquet column chunks load into VertiPaq column segments
- Per column, on demand

Columns needed by the query!



Transcoding – Loading columns into memory

- On first touch, parquet column chunks load into VertiPaq column segments
- Per column, on demand
- Columns loaded into memory -> Resident

Reasons to remove a resident column

- The model or table was refreshed after a Delta table update at the source
- No query used the column for some time
- Other memory management reasons, including memory pressure in the capacity due to other, concurrent operations

The residency ladder

The mental model for every demo today

Cold

Nothing resident. Every query pays full transcoding.

Semiwarm

Only changed row groups reload. Also the memory-pressure state.

Warm

Queried columns resident. Import-grade speed.

Hot

VertiScan caches populated. Queries hit the cache.

Every demo: which rung are we on, and what pushed us off it?

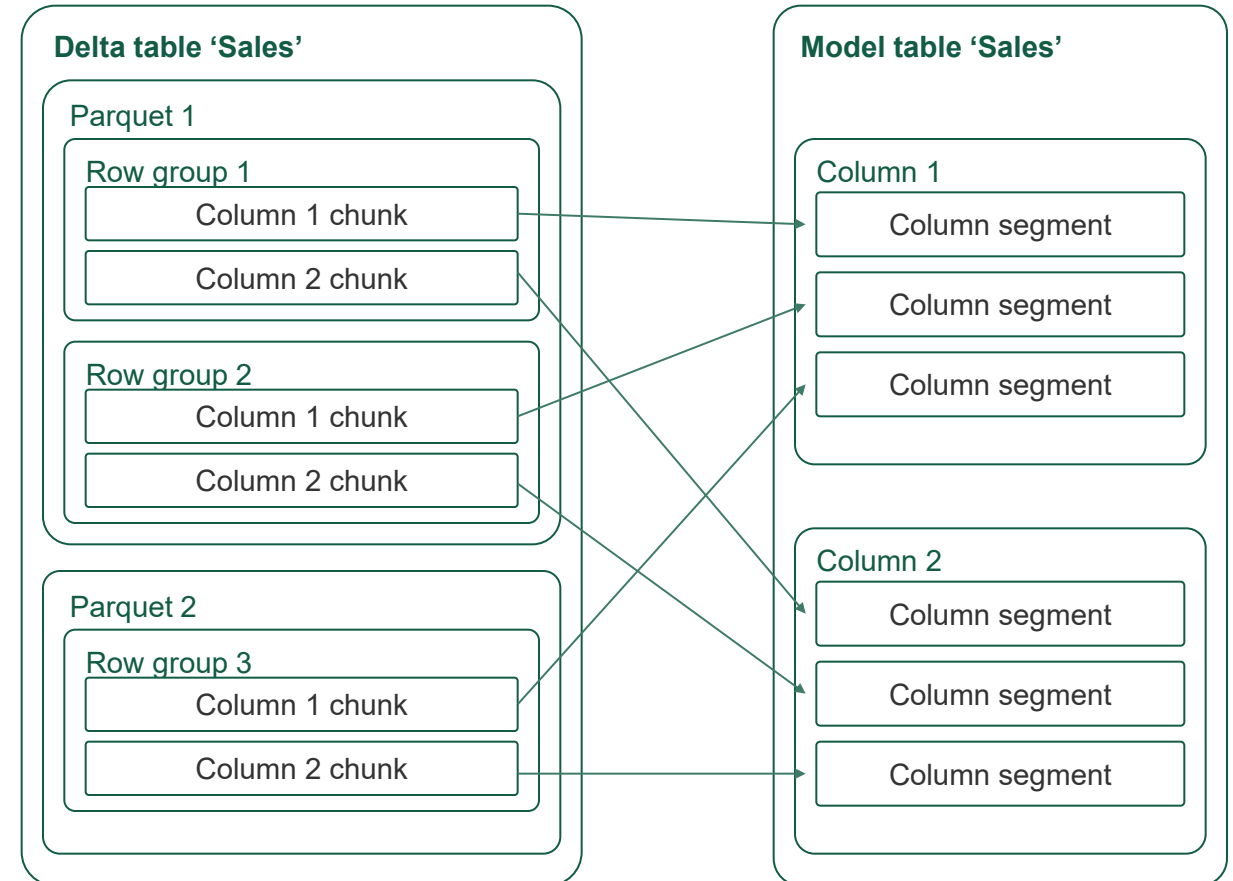
Row groups and column segments

One to one. Total row groups in the Delta table = segments per column in the model.

Target size. 1 to 16 million rows per row group. Tiny row groups mean tiny segments and slow scans.

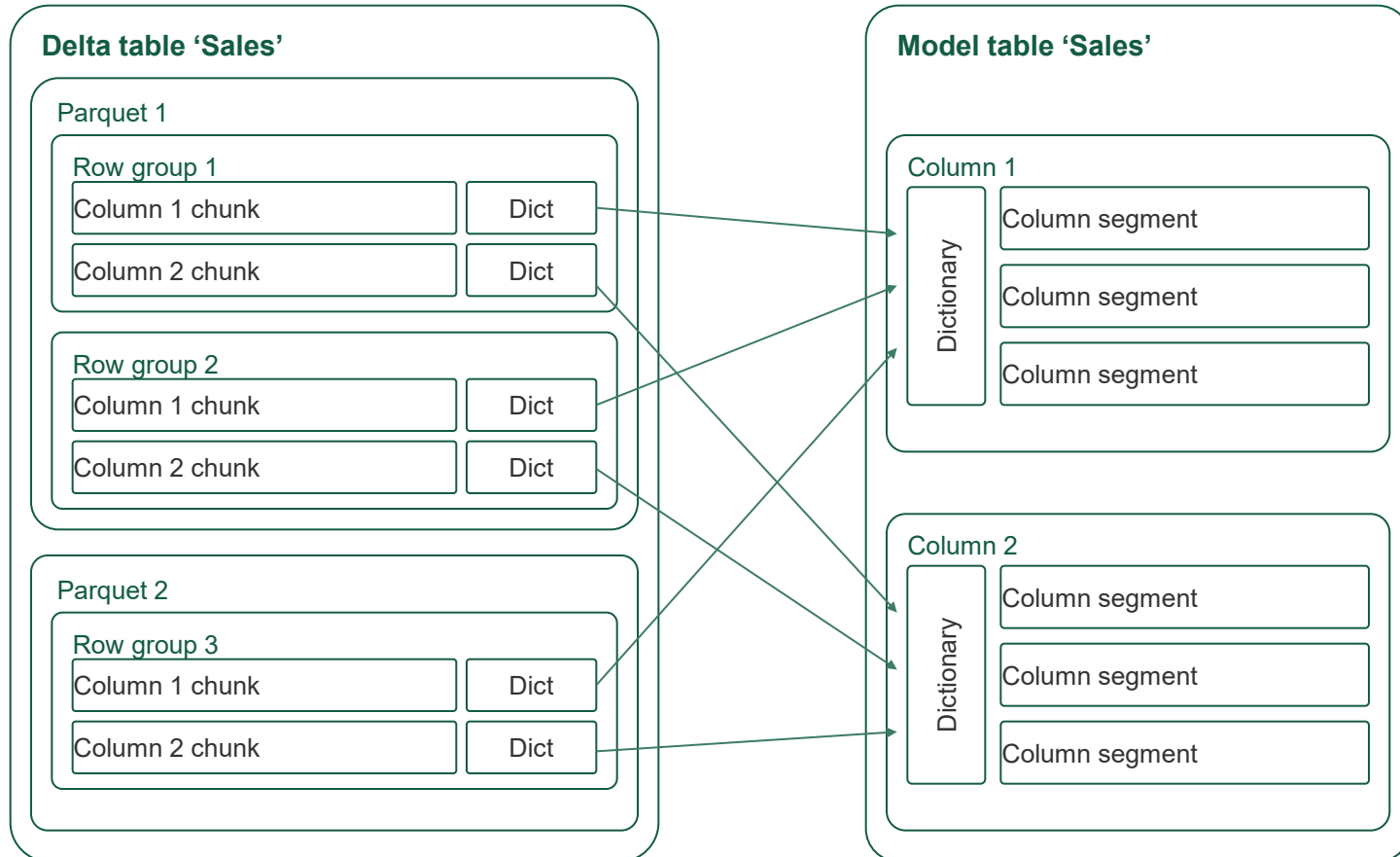
Uniformity. Uneven row group sizes cause uneven VertiScan load.

Who decides? Your write patterns and OPTIMIZE cadence. Fabric manages layout well when you let it.



Measured 3,392 row groups on the same 8 files: 4.8x slower cold than 16 row groups. At 5,552 row groups the table exceeded the F256 guardrail and would not load.

Transcoding is a dictionary merge



Every row group carries its own local parquet dictionary per column.

Direct Lake merges all of them into one global dictionary before the column can answer.

More row groups, longer merges. Fragmentation taxes transcoding twice.

V-Order helps: tightly packed RLE data maps almost directly into VertiPaq.

Measured V-Order off, identical layout: 3.6x slower cold on numeric columns, 5.8x once string columns are involved. Same segments resident at the end.

Incremental framing

- DL analyzes the Delta log of each Delta table -> drops loaded segments and join indexes **only when the underlying data has changed**
- **Dictionaries** retained -> new values simply added to existing dictionaries
- Reduces the reload burden and helps cold query performance

Delta table update patterns

- Overwrite – truncates the Delta log -> **NO INCREMENTAL FRAMING!!!**
- Batch processing without vs. with **partitioning**
- **Incremental loading**
- **Stream processing** -> many small files/row groups -> worse performance

Delta table maintenance

- **VACUUM** - removes unreferenced Parquet files -> **doesn't affect DL performance!**
- **OPTIMIZE** – merges small files into larger ones -> **affects cold DL performance!**
 - Optimize infrequently
 - Optimize more frequently if small Parquet files accumulate quickly
- Ideal row group size = **1 – 16 million rows** -> **don't change the default size!**

Append-only pattern + OPTIMIZE

The pruning myth

Direct Lake doesn't skip files or row groups.

No partition pruning. No min/max skipping. Transcoding reads what the column needs - period.

Measured part_month 1.16s vs base 0.80s (order flips between runs). part_date 7.86s: 1,826 small files. Partitioning changes nothing on read unless it fragments the table.

So partitioning is useless?

For query speed, yes: your Spark-era pruning instincts don't transfer.

No: it earns its keep elsewhere

Containing the blast radius of deletes, merges, and OPTIMIZE. Stay tuned 😊

Source: Understand Direct Lake query performance, learn.microsoft.com

PART 2 - LIVE DEMO

Layout: files, row groups, V-Order, partition reads

One baseline, one variable per variant

Mosaic ticket sales - 500 million rows in every table, identical schema, one thing changed each time

Variant	What is different	Files	Row groups	Rows per row group
base	compacted, V-Order on, unpartitioned	8	16	31.3M
many_files	file count only, healthy row groups	500	500	1.0M
small_rowgroups	row group size only (compare with novorder)	8	3,392	147K
fragmented	everything wrong: tiny files, tiny row groups, no V-Order	4,000	4,000	125K
novorder	V-Order off, otherwise identical to base	8	16	31.3M
part_month	partitioned by month_key (60 values)	60	60	8.3M
part_date	partitioned by date_key (1,826 values)	1,826	1,826	274K

Every comparison that follows changes exactly one row of this table against another.

The queries behind every number

Three reference queries against the same fact table, plus the wrapper that keeps the result cache out of the measurement

Light

3 columns. Cold 0.8s, warm 0.3s on base. Used for every layout comparison.

```
EVALUATE
SUMMARIZECOLUMNS(
  dim_event[event_category],
  "revenue", SUM(fact[gross_amount]),
  "tickets", SUM(fact[quantity])
)
```

Medium

7 columns, no distinct count. Cold 0.8s on base, 4.6s without V-Order. Used for residency.

```
EVALUATE
SUMMARIZECOLUMNS(
  fact[venue_city],
  fact[ticket_type],
  fact[payment_method],
  "revenue", SUM(fact[gross_amount]),
  "tickets", SUM(fact[quantity]),
  "avg_price", AVERAGE(fact[unit_price]),
  "orders", COUNTROWS(fact)
)
```

Heavy

Adds a distinct count over 2M values. 127s cold on base. Cardinality beats layout.

```
EVALUATE
SUMMARIZECOLUMNS(
  fact[venue_city],
  fact[ticket_type],
  fact[payment_method],
  dim_date[year],
  "revenue", SUM(fact[gross_amount]),
  "tickets", SUM(fact[quantity]),
  "avg_price", AVERAGE(fact[unit_price]),
  "customers", DISTINCTCOUNT(fact[customer_key])
)
```

Cache bypass, every timed query

n increments on each call

```
EVALUATE CALCULATETABLE(<query>, dim_event[event_key] <> -n)
```

Your dev dataset hides this

10 million rows

well shaped **0.81s**

fragmented **1.50s**

1.9x slower cold

Fragmented looks fine. Nobody complains.

500 million rows

well shaped **0.80s**

fragmented **13.99s**

17.5x slower cold

Same layout, same query. **The table is now the problem.**

Cold light query, cache bypassed, medians from the evidence run. Warm: 0.33s and 0.36s at 10M, 0.32s and 0.49s at 500M.

Part 2 results - cold transcoding time by variant

Light and medium queries, cold after clearValues, cache bypassed, medians of three.

Variant	Files	Row groups	V-Order	Cold, light	Cold, medium	vs base (light)
base	8	16	true	0.80s	0.78s	1.0x
many_files	500	500	true	2.68s	3.57s	3.4x
small_rowgroups	8	3,392	false	13.70s	18.98s	4.8x vs novorder
fragmented	4,000	4,000	false	13.99s	21.43s	17.5x
novorder	8	16	false	2.86s	4.56s	3.6x / 5.8x
part_month	60	60	true	1.16s	1.33s	1.5x (noise)
part_date	1,826	1,826	true	7.86s	11.74s	9.8x

Warm, every variant: 0.32 to 0.50s. The whole penalty is transcoding. Once the segments are resident, the layout stops mattering.

Part 2 takeaways

Files

17.5x

slower cold

4,000 files vs 8. 500 files already cost 3.4x with perfectly healthy row groups.

Row groups

4.8x

slower cold

3,392 row groups vs 16 on byte-identical files. The 5,552 version exceeded the F256 guardrail and would not load.

V-Order

3.6x

slower cold

One table property, same files, same segments resident at the end. 5.8x once string columns are involved.

Partitions

9.8x

slower cold

By date key: 1,826 small files. By month key: no read benefit, no read cost. Direct Lake never prunes.

OPTIMIZE and V-Order are query-performance features, not housekeeping.

PART 3 - LIVE DEMO

Partition writes, write patterns, residency

Blast radius of a one-day delete

DELETE WHERE date_key = 20210101 - 274,138 rows, 0.05 percent of the table - then reframe

Table	Files before	Files removed	Files added	Warm segments before -> after	Retained
base	8	8	16	128 -> 0	0%
part_month	60	1	1	480 -> 473	99%
part_date	1,826	1	0	14,608 -> 14,600	100%

0.05% of rows deleted. 100% of the cache gone.

Parquet files are immutable: a delete rewrites every file that held a matching row. Unpartitioned, that was all of them. Partitioned by month, one file. The high-cardinality partition removed its file and added none - the only thing it does well.

Every write is a cache decision

Same table, same warm state before each write: 112 segments across 7 columns resident. After reframing:

Append

100% retained

New files only. 0 removed, 32 added.

Every warm segment survived the reframe. Only the new rows are cold.

But: 550K rows became 32 row groups of 17K. Appends without OPTIMIZE are how fragmented tables are born.

Merge, 1% of rows

0% retained

Rewrites every file holding a matched row: 8 removed, 33 added.

0 of 396 segments resident afterwards.

Compaction helps: fewer files rewritten per matched row.

Overwrite, same data

0% retained

Logically a no-op. Physically 8 removed, 17 added.

0 of 192 segments resident afterwards.

Delta records it as CREATE OR REPLACE TABLE AS SELECT - a nightly full load, literally.

Your orchestration pattern is a performance decision, not just a correctness decision.

The instruments

Everything in this session is measured with three things you already have

Delta side

`delta_analyzer()`

semantic-link-labs. Files, row groups, rows per row group, V-Order flag - the layout facts per table.

Write side

DESCRIBE HISTORY

Spark. `numRemovedFiles` and `numAddedFiles` per operation - the blast radius of every write, one line per commit.

Model side

INFO.STORAGETABLECOLUMNSEGMENTS()

DAX. Per column segment: `DIMENSION_NAME`, `ISRESIDENT`, `TEMPERATURE`. What is in memory right now, and how warm.

Sequence, every time: cold reset > warm > write > reframe > residency > timing, with the result cache bypassed

Part 3 takeaways

Append survives. Merge dies. Overwrite always dies.

100% of warm segments retained after an append. 0% after a 1 percent merge. 0% after an overwrite of identical data.

Partition for blast radius, never for pruning

One file rewritten instead of eight. Pick a key with fewer than 200 values: month, not date.

Direct Lake loads per column, warms per access

A single-column query loaded one column. Temperature climbed 0.97, 1.87, 2.76 as the same columns were hit again. Nine columns never loaded.

Residency, measured

INFO.STORAGETABLECOLUMNSEGMENTS() on the base table, 16 segments per column

After a one-column query

Column	Segments	Resident	Temp.
gross_amount	16	16	0.97
event_key	16	16	0.97
quantity	16	0	-
venue_city	16	0	-
ticket_type	16	0	-
payment_method	16	0	-
unit_price	16	0	-
5 more columns	16	0	-

Then after a seven-column query

Column	Segments	Resident	Temp.
gross_amount	16	16	1.90
event_key	16	16	1.90
quantity	16	16	0.97
venue_city	16	16	0.97
ticket_type	16	16	0.97
payment_method	16	16	0.97
unit_price	16	16	0.97
5 more columns	16	0	-

event_key loaded with the first query because the filter runs through the dim_event relationship - relationship columns always load. Columns touched twice are warmer than columns touched once. Columns nobody queries cost nothing.

Partitioning - Wrong reason, right tool

Not for pruning

Direct Lake ignores partitions at query time. Partitioning will not make a single DAX query faster.

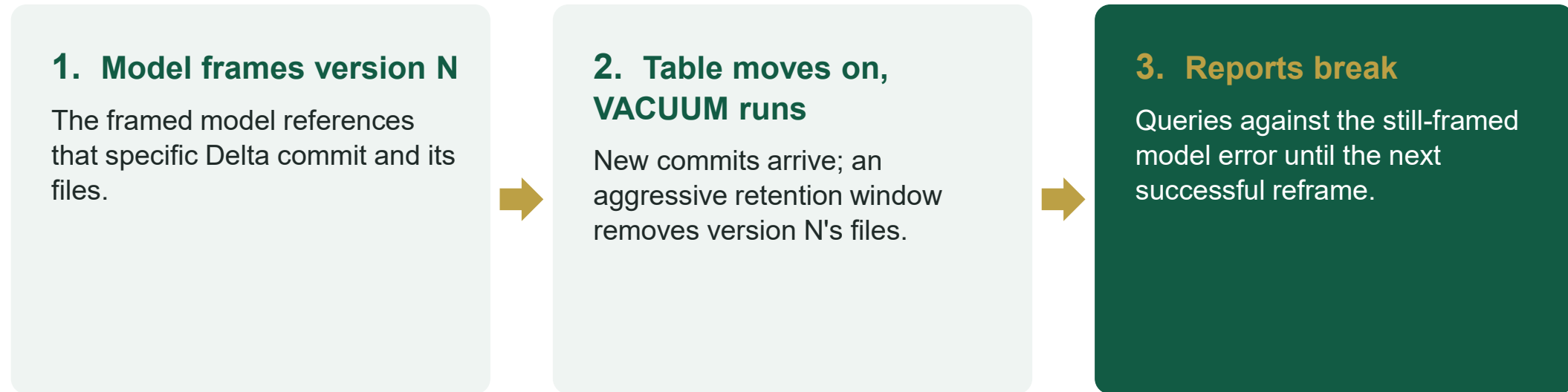
For blast radius

Deletes, merges, and OPTIMIZE touch only the affected partitions. Incremental framing keeps the rest warm: a rolling-window delete reloads one file instead of all of them.

Rules: keep the partition column under 100 to 200 distinct values.

A raw date key fails that test (1,825 values over 5 years). Derive a month key instead: 60 values, and weekly maintenance touches at most a handful of partitions.

The VACUUM trap



Rule: retention must cover your reframe gap.

Default 7-day retention is safe with daily refresh. Aggressive retention plus a slow refresh cadence is how visuals break on a Monday.

Two (in)visible costs

Memory pressure

Everything queried must fit in memory as segments and join indexes.

Under pressure, segments unload: back down the ladder to semiwarm.

Watch: wide tables, high-cardinality strings, capacity sizing.

The fallback tax (DL-SQL only)

Fallback support forces hybrid query plans.

You pay the overhead even when no query ever falls back.

If you don't need it, disable DirectQuery fallback.

Measured A DISTINCTCOUNT over a 2M-value column turned a 0.78s cold query into 127s on the best-laid-out table in the matrix. The column you query can cost more than the files under it.

The checklist

DESIGN

Right-size data types; integers over strings, trim cardinality.

Partition only with a sub-200-value column; derive month, never raw date.

WRITE

Append where possible; keep merges narrow and targeted.

Never fully overwrite a large fact as a routine load.

MAINTAIN

OPTIMIZE with V-Order on a cadence matched to write frequency.

VACUUM retention must cover the reframe gap.

VERIFY

Delta Analyzer for files and row group sizes.

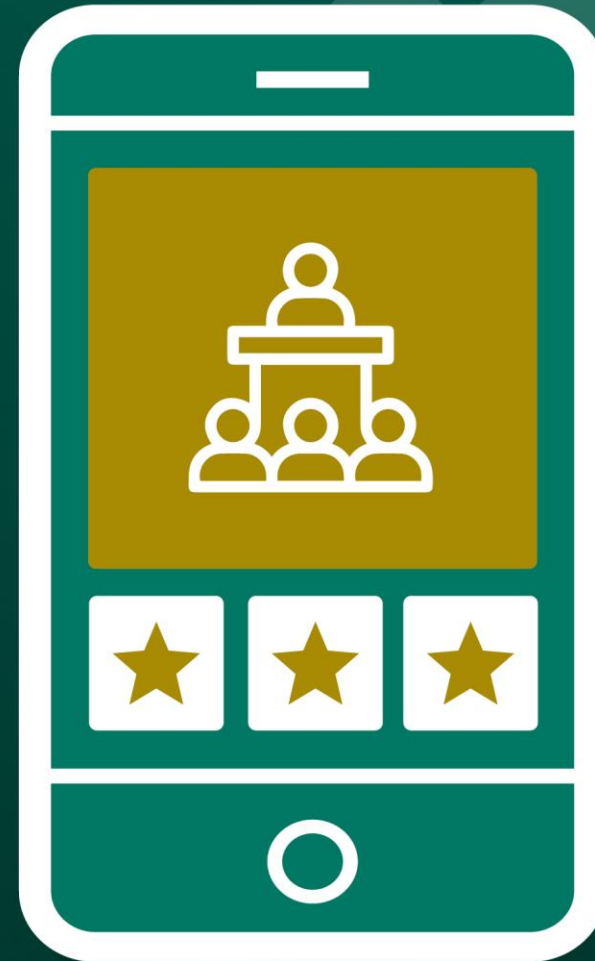
Segment residency via
INFO.STORAGETABLECOLUMNSEGMENTS, cold and warm,
at production scale.

[Reference: Understand Direct Lake query performance](#)

Thank you!



Please rate
this session
on the app



cvent

